University of Technology, Jamaica
School of Computing and Information Technology
Advanced Programming (CIT3009)

Advanced Programming Group Project

University Help Desk & Ticketing System

Given: October 2, 2026 Due: November 20, 2026 Group size: 4 (Maximum)

Project Overview

The University Help Desk & Ticketing System is a distributed software solution designed to support the reporting, tracking, assignment, and resolution of technical support issues within a university environment. The system will provide a desktop-based JavaFX client for end users and support personnel, a RESTful web service layer developed using Spring Boot, a relational database for persistent storage, a TCP/IP notification service for real-time communication, and robust logging and exception handling mechanisms.

The project is intended to serve as a final assessment for Advanced Programming and requires students to collaboratively integrate all major concepts covered throughout the course into a single coherent application.

Project Scenario

The university's IT Services Department currently receives support requests via email, phone calls, and walk-in reports. This process is inefficient and makes it difficult to:

- Track reported issues
- Assign technicians
- Monitor resolution progress
- Generate reports
- Notify users of status changes
- Maintain a historical record of incidents

The university has commissioned your development team to design and implement a computerized Help Desk & Ticketing System that addresses these issues.

Project Objectives

Students will design and develop a software solution capable of:

- Recording support tickets.
- Managing users and technicians.

- Assigning tickets to technicians.
- Tracking ticket progress.
- Exchanging information through RESTful services.
- Storing data persistently within a relational database.
- Providing real-time notifications through network communication.
- Maintaining audit logs.
- Processing concurrent requests safely.

Intended Users

The system must support the following user roles:

Student / Staff User

Can:

- Submit support requests.
- View submitted tickets.
- View ticket status.
- Add comments to tickets.
- Receive notifications.

Technician

Can:

- View assigned tickets.
- Update ticket status.
- Add resolution notes.
- Mark tickets as resolved.

Administrator

Can:

- View all tickets.
- Manage users.
- Manage technicians.

- Assign tickets.
- Generate reports.
- Monitor system activity.

Functional Requirements

User Management

The system shall allow:

- User registration.
- User login/logout.
- Password management.
- Role assignment.

Ticket Management

Users shall be able to:

- Create tickets.
- View tickets.
- Update ticket details.
- Close tickets.

Each ticket shall contain:

- Ticket ID (Auto-generated)
- Subject
- Description
- Creation Date
- Priority
- Status
- Assigned Technician
- Resolution Notes

Status Tracking

The following statuses must be supported:

- NEW
- ASSIGNED
- IN_PROGRESS
- PENDING
- RESOLVED
- CLOSED
- REOPENED

Search Functionality

The system shall support searching by:

- Ticket ID
- User
- Technician
- Status
- Priority
- Date Range

Reporting

Generate reports such as:

- Tickets Submitted
- Tickets Resolved
- Open Tickets
- Technician Workload
- Average Resolution Time

Technical Requirements

---

1. JavaFX GUI Development

Students must develop a graphical desktop application using JavaFX.

Required Screens

1. Login Screen
2. User Dashboard
3. Technician Dashboard
4. Administrator Dashboard
5. JavaFX Concepts

Students must demonstrate:

- Stage
- Scene
- Layout Panes
- Event Handling
- Alerts
- TableView
- Form Validation
- MVC Pattern

---

2. RESTful Service Development

The backend shall be implemented using Spring Boot.

Required Endpoints

- User Endpoints
- Ticket Endpoints
- Technician Endpoints
- Reports Endpoints

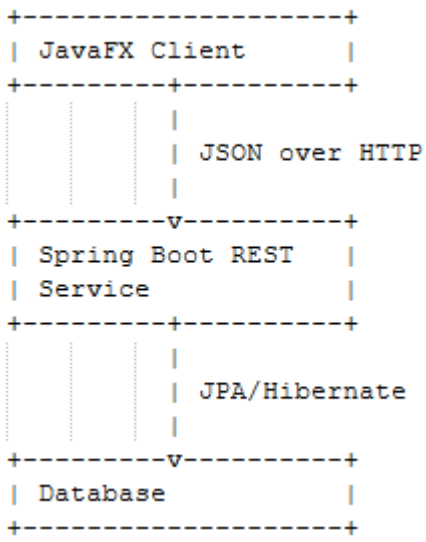
---

3. Data Persistence & Exchange

The application must exchange data using JSON.

Example:

The information exchanged between:



The exchanged information must be persisted permanently in the database.

---

4. Database Connectivity & ORM

Students shall use:

- JPA
- Hibernate ORM
- MySQL

Minimum Database Entities

- User
- Technician
- Ticket
- Comment

Required Relationships

Students must demonstrate:

User | | One-to-Many | Ticket	Technician | | One-to-Many | Ticket	Ticket | | One-to-Many | Comment

---

5. Inversion of Control & Dependency Injection

Students must implement Dependency Injection throughout the application.

Required annotations:

@Component

@Service

@Repository

@Controller

@Autowired

The design should follow a layered architecture:

---

6. Exception Handling

The system must handle errors gracefully.

Custom Exceptions

Students must create at least three custom exceptions.

Examples:

1. TicketNotFoundException
2. UserNotFoundException
3. TicketAssignmentException

GUI Error Handling

Messages displayed using JavaFX Alerts.

Examples:

- Ticket Not Found
- Invalid Login Credentials
- Unable to Connect to Server
- Duplicate User Account

---

7. Logging with Log4j2

Every significant operation must be logged.

Examples:

- User Login
- User Logout
- Ticket Creation
- Ticket Assignment
- Ticket Resolution
- Database Errors
- REST API Errors
- Notification Failures
- Unhandled Exceptions

Information Logs

```
logger.info("Ticket created");
```

Warning Logs

```
logger.warn("Invalid login attempt");
```

Error Logs

```
logger.error("Database connection failed");
```

Logging Requirements

Students must implement and configure:

1. Application Log File
2. Error Log File
3. Rolling Log Files

---

8. Multithreading & Concurrency

The application must use multiple threads responsibly.

Requirement 1:

Background Ticket Refresh

The technician dashboard must automatically refresh ticket information without freezing the GUI.

Requirement 2:

Concurrent Ticket Assignment

The system must prevent multiple administrators from assigning the same ticket simultaneously.

Requirement 3:

Notification Processing

Notifications should be processed on background threads.

---

9. TCP/IP Client-Server Networking

In addition to REST services, the system must include a separate notification server.

Notification Server

The Notification Server shall:

- Accept client connections.
- Receive ticket updates.
- Broadcast notifications.
    - Ticket Assigned
    - Ticket Resolved
    - New Comment Added
    - Urgent Ticket Created
- Maintain connected client sessions.

Suggested Team Structure

Team Member 1

Front-End Developer

Responsible for:

- JavaFX Views
- Navigation
- Input Validation
- Alerts

Team Member 2

Backend Developer

Responsible for:

- REST Controllers
- Service Layer
- Data Transfer Objects (DTOs)

Team Member 3

Data & Persistence Developer

Responsible for:

- Database Design
- JPA Entities
- Repository Layer
- Queries

Team Member 4

Systems Developer

Responsible for:

- Multithreading
- Socket Programming
- Log4j2
- Exception Handling

Deliverables

Each group must submit:

Source Code

- JavaFX Application
- Spring Boot Application
- Notification Server

Documentation

- Problem Definition
- Requirements Analysis
- Class Diagram
- ER Diagram
- Use Case Diagram
- System Architecture Diagram

Demonstration

A live demonstration (by all members) showing:

- User Login
- Ticket Creation
- Database Persistence
- REST API Communication
- Notification Delivery
- Logging
- Exception Handling
- Multithreading Features
- Socket Communication

Marking Guide (100 Marks)

Component	Marks
JavaFX GUI & MVC	12
RESTful Service Implementation	12
Database & ORM	12
TCP/IP Networking	12
Multithreading & Concurrency	10
Dependency Injection & IoC	10
Exception Handling	10
Log4j2 Logging	5
Documentation	3
Demonstration & Presentation	14

Total: 100 Marks

Absence from project presentation incurs an automatic zero

Late submission

As per the university's policy, late submission of projects will attract a penalty as follows:

Time of Submission after Deadline	Penalty (%)
One Day (or any part thereof)	10
Two Days (or any period > 1 day, but <= 48 hrs.)	20
Three Days (or any period > 2 days, but <= 72 hrs.)	50
Beyond Three Days	100

Expected Outcome

Upon completion, students will have developed a multi-tiered enterprise-style application that demonstrates modern Java software development practices, integrating desktop GUI development, RESTful web services, ORM-based persistence, networking, concurrency, dependency injection, logging, and robust exception handling into a cohesive solution.